

Algorithmic Analysis of SARS-CoV-2 Sequences

Advanced Algorithms Design
TC2038, Situation Problem 1

Santiago Hernández
A01787550

September 21, 2026

Contents

1 Problem definition **2**

2 Datasets **2**

3 Algorithm **3**
3.1 Longest common subsequence by dynamic programming 3
3.2 Hirschberg: divide and conquer on top of the recurrence 4
3.3 From subsequence to mutations 4
3.4 Supporting techniques 4

4 Implementation **5**

5 Complexity analysis **6**

6 Results **7**
6.1 Genome comparison 7
6.2 Search, greedy, backtracking, branch and bound, palindromes 8
6.3 Performance 9

7 Conclusions **10**

References **10**

Appendix **12**

A Additional tables **12**

B Additional figures **14**

1 Problem definition

The assignment explores how different algorithmic strategies can analyse, search, compare and characterise biological sequences. Rather than apply one technique, this report applies every technique of the course to a concrete question that the problem scenario poses about the SARS-CoV-2 files, and then compares the techniques on measured cost. The main technique of the report is Hirschberg's algorithm, dynamic programming for the longest common subsequence combined with divide and conquer. It is applied to the comparison of the Wuhan 2019 genome (MN908947.3) with the Texas 2020 genome (MT106054.1). The other techniques are supporting evidence that answer smaller questions about the same files and provide the baselines against which the main result is judged.

The mapping between questions and techniques is:

- Where are the genes M, S and ORF1ab inside the genome? Exact pattern search: brute force as the baseline, then KMP and the Z-function.
- How similar are the two genomes and what exactly changed? Longest common subsequence by dynamic programming, made feasible for two full genomes by Hirschberg's divide-and-conquer formulation.
- Which open reading frames form the largest non-overlapping set? Greedy activity selection.
- Where is each protein of `seq-proteins.txt` encoded? Backtracking over codon choices with pruning by the genome.
- Which motif of length k occurs most often when one mismatch is allowed? Branch and bound over the tree of prefixes.
- Which region of each gene is the longest palindrome? Manacher's algorithm against centre expansion (extra technique).

Success is measured by three criterion per technique: a correct answer checked against an independent computation, a justification of why the technique fits the problem, and a measured running time compared with its theoretical complexity and with a brute-force baseline.

2 Datasets

All six files are in FASTA format: a header line beginning with `>` followed by lines of sequence. The gene files carry their coordinates inside the reference genome in the header (for example `MN908947.3:21563-25384` for gene S), this gives an independent ground truth for the search algorithms.

Every file is read the same way. First, lines are stripped of whitespace and carriage returns (the DNA files use Windows line endings), the header line is skipped, and the remaining symbols are then uppercased. Any symbol outside {A, C, G, T} for DNA, or outside the 20 amino-acid letters for proteins, is counted and reported as unexpected. The check found no unexpected symbol in any of the six files (Table 1).

file	length	symbols	most	least	unexpected
SARS-COV-2-MN908947.3.txt	29903	4	T	C	none
SARS-COV-2-MT106054.1.txt	29882	4	T	C	none
gen-ORF1AB.txt	21290	4	T	C	none
gen-S.txt	3822	4	T	G	none
gen-M.txt	669	4	T	G	none
seq-proteins.txt	9701	20	L	W	none

Table 1: Characterisation of the six files. Symbols counts the distinct expected symbols found.

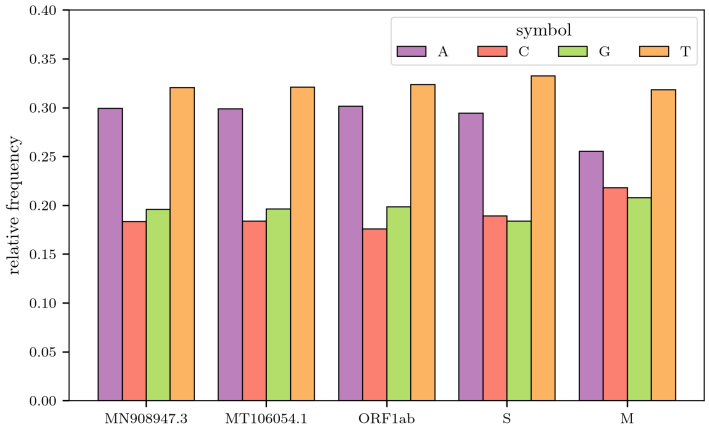


Figure 1: Relative nucleotide frequencies per DNA file.

T is the most frequent base in every file. C is the least frequent in both genomes and in ORF1ab, while in S and M the least frequent base is G. Gene M is the least skewed of the five DNA files. The two genomes have practically identical composition (A 0.299, C 0.184, G 0.196, T 0.321 in both), a first hint of how close they are before any alignment is computed. The amino-acid frequencies (Figure 8 in the appendix) are far from uniform: L is the most frequent and W the least.

3 Algorithm

3.1 Longest common subsequence by dynamic programming

For sequences $X = x_1 \dots x_n$ and $Y = y_1 \dots y_m$, the length of the longest common subsequence of the prefixes $x_1 \dots x_i$ and $y_1 \dots y_j$ satisfies

$$L(i,j) = \begin{cases} 0 & i = 0 \text{ or } j = 0, \\ 1 + L(i-1, j-1) & x_i = y_j, \\ \max\big(L(i-1,j),\, L(i,j-1)\big) & x_i \neq y_j. \end{cases} \tag{3.1}$$

The recurrence has overlapping subproblems. Each $L(i,j)$ is needed by up to three other cells, so the naive recursion recomputes the same values exponentially many times. An optimal solution for the pair (i,j) contains an optimal solution for a smaller pair, which is what makes the recurrence valid. A table filled row by row therefore costs $\Theta(nm)$ (Cormen et al., 2009). The subsequence itself is recovered by tracing back through the table.

For the two genomes the table has $29\,903 \times 29\,882 \approx 8.9 \times 10^8$ cells, which is 3.6 GB at 4 bytes per cell, so the textbook algorithm cannot be run on a laptop or in Colab. The length alone needs only two rows at a time, because a row depends only on the previous one. The traceback, however, needs the whole table, so discarding the table also discards the alignment.

3.2 Hirschberg: divide and conquer on top of the recurrence

Hirschberg's algorithm removes the table without giving up the alignment (Hirschberg, 1975). It rests on three facts. First, the last row of the table, $L(\text{mid}, \cdot)$, can be computed with two rows of memory. Second, for the split of X at its midpoint, $|LCS(X,Y)| = \max_k \big(|LCS(X[:\text{mid}], Y[:k])| + |LCS(X[\text{mid}:], Y[k:])|\big)$, and the maximising k is found by adding the forward last row of $X[:\text{mid}]$ against Y to the last row of the reversed $X[\text{mid}:]$ against the reversed Y. Third, once k is fixed the two halves are independent, so they are solved recursively and their results concatenated. Divide is the split of X, while conquer is the recursion on the two sub-rectangles and combine is the concatenation. Memory is $O(n+m)$ and time stays $O(nm)$, because the rectangles shrink geometrically (Section 5).

Subproblems with at most 40 000 cells are solved with a full table and a traceback, which is the base case of the recursion. On the two genomes the recursion produced 255 split points with a maximum depth of 7 (Figure 5 in the appendix). A single-symbol X is handled as its own base case, so the split index is always at least 1 and every call makes progress.

3.3 From subsequence to mutations

Consecutive matched pairs (i, j) and (i', j') delimit an unmatched gap, $X[i+1 : i']$ against $Y[j+1 : j']$. Because the LCS is optimal, the two sides of a gap share no symbol at all: a shared symbol could be matched without disturbing any other pair, and the LCS would grow. A gap with equal length on both sides is therefore a run of substitutions, a gap with one empty side is an insertion or a deletion, and anything else is a complex change.

Each substitution inside a coding region is translated in both genomes with the standard genetic code and classified as synonymous or non-synonymous. The reading frames are not assumed. Gene S and gene M translate exactly to their protein records, and ORF1ab is translated in two frames because of the ribosomal frameshift at position 13 468: the ORF1a frame from 266 and the ORF1b frame from 13 468. Ten protein records are found whole in the first frame and four in the second, which validates both the frames and the codon table. Once backtracking (below) locates every protein, the classification is repeated over all coding regions.

3.4 Supporting techniques

Naive search tries every alignment and compares symbol by symbol, which is $O(nm)$ in the worst case. KMP preprocesses the pattern into its failure table, so that after a mismatch the text pointer never moves backwards (Knuth et al., 1977). The Z-function computes, for `pattern + # + text`, the length of the longest prefix match at every position, and a value equal to m is an occurrence. Both are $O(n+m)$. The improved methods avoid the repeated work of naive search because after a mismatch they reuse what is already known about the pattern instead of restarting the comparison.

We consider every open reading frame of at least 100 codons in the three forward frames, each running from an ATG to the next in-frame stop. Picking the largest set of these that do not overlap is the activity selection problem. The greedy rule sorts candidates by end position and keeps each one that does not overlap the last one kept. This is optimal (CLRS ch. 16): the earliest-ending interval can always replace the first interval of an optimal solution without creating an overlap. The limitation appears in the results: the objective is the count of intervals, not the coverage of the genome.

The genetic code has up to six codons per amino acid, so a protein of length L has up to 6^L encodings. The search builds an encoding codon by codon. The state of a node is the set of genome positions still consistent with the codons chosen so far, and a choice that leaves that set empty is pruned together with its whole subtree. Only the first $k = 6$ amino acids are searched this way. Every surviving position is then verified by translating the genome forward for the whole protein.

The candidates are k-mers built symbol by symbol, so the search space is a tree of prefixes. A node carries the genome positions whose window still agrees with the prefix within d mismatches, and the number of those positions is a valid upper bound on the count of any completion of the prefix, because a window within d mismatches of the full k-mer is within d mismatches of its prefix. Nodes are expanded best first from a max-heap ordered by that bound, the incumbent comes from a greedy descent, and a node is pruned when its bound cannot beat the incumbent. A hash table is not used because with $d = 0$ a single pass over the genome solves the problem in $O(nk)$, which is why the mismatch version is the one that justifies branch and bound.

Centre expansion tries the $2n-1$ possible centres and is $O(n^2)$ in the worst case. Manacher's algorithm interleaves a separator so that every palindrome has odd length, then reuses the radius of the mirror centre inside the rightmost known palindrome (Manacher, 1975). Each symbol extends the right boundary at most once, so the algorithm is $O(n)$.

4 Implementation

A single notebook, `dna_analysis.ipynb`, runs from top to bottom in Jupyter or in Colab, and needs only numpy and matplotlib. Every string algorithm is written from scratch: no `str.find`, no library search and no library alignment. Numpy is used only to vectorise a dynamic programming row and for plotting. The notebook writes every figure and every table of this report into `figures/` and `tables/`.

The main components are: `read_fasta` and `characterize` (parsing, Part 1 statistics, the unexpected-symbol check); `naive_search`, `kmp_search` with `build_lps`, `z_search` with `z_function`, and `search_all`, which runs the three and asserts that they agree; `lcs_table` (full table with traceback, the oracle), `lcs_rows_py` (two rows, pure Python), `lcs_last_row` (two rows, numpy) and `hirschberg`, which returns the matched index pairs; `alignment_events`, `classify_substitutions` and `translate` with the standard codon table; `find_orfs`, `greedy_select`, `exhaustive_select`; `locate_backtracking`, `locate_bruteforce` and `matched_prefix` on top of a codon index (a dictionary from every 3-mer to its positions); `motif_bnb` and `motif_exhaustive` sharing `motif_children`; `longest_palindrome_center` and `manacher`; and `bench` (minimum of repeated runs with `perf_counter`) with `loglog_slope`.

A row of the LCS table is a running maximum. When $x_i = y_j$ the diagonal value plus one dominates both neighbours. Otherwise, the cell is the maximum of its two neighbours. So with $c_j = L(i-1, j-1) + 1$ if $x_i = y_j$ and $c_j = L(i-1, j)$ otherwise, the row is $L(i, j) = \max(c_j, L(i, j-1))$, a cumulative maximum that numpy computes with `maximum.accumulate` in one vectorised call per row. This is what makes Hirschberg run on two full genomes in a few seconds of Python.

The three search methods return identical positions on every search, and the gene positions equal the header coordinates minus one. The four LCS implementations (full table, two rows in Python, two rows in numpy, and Hirschberg) agree with each other and with an independent bit-parallel oracle: on the textbook example, on 200 random strings with the recursion forced down to tiny base cases, and on four genome windows containing substitutions, shifts and unequal lengths. On the full genomes the Hirschberg pairs form a valid common subsequence whose length equals the oracle's. Gene S and gene M translate exactly to their protein records. Greedy equals the exhaustive optimum on 14 candidates spread over the genome. Backtracking and brute force find the same positions for every protein. Branch and bound equals exhaustive

search at $k = 8$, and with $d = 0$ it equals a hash count. Manacher equals centre expansion on the three genes.

5 Complexity analysis

Filling the table is $\Theta(nm)$ in time and $\Theta(nm)$ in space. Computing the length alone is still $\Theta(nm)$ in time but only $\Theta(m)$ in space. For the two genomes that is 8.9×10^8 cells: 3.6 GB as a table against 239 KB as two rows of 32-bit integers.

At the top level the two last-row computations cost nm cell updates. At the next level the two sub-rectangles have total area at most $nm/2$, because the split cuts both X and Y. The areas form a geometric series, so the total is at most $2nm$ cell updates, that is, $O(nm)$ time with $O(n + m)$ memory, and the depth is $O(\log n)$. The constant is worth stating: about twice the work of the length-only version, which is what the measurements show, 3.79 s against 1.68 s on the full genomes in the benchmark run (Table 4).

Naive search is $O(nm)$ in the worst case, reached by the adversarial text A^n with the pattern $A^{m-1}C$, where every alignment compares all m symbols. On real DNA a mismatch occurs after about 1.3 comparisons on average, the expected value for a four-letter alphabet, so naive search behaves linearly. KMP is $O(m)$ preprocessing plus an $O(n)$ scan, because the text pointer only advances and the pattern pointer decreases at most as often as it increases. The Z-function is $O(n + m)$ by the same amortised argument applied to the right boundary.

Greedy ORF selection costs $O(n)$ to scan the three frames and $O(c \log c)$ to sort the c candidates, against $O(2^c)$ for the exhaustive subset search, whose doubling per candidate is visible in Figure 4. The worst case of backtracking is the number of encodings, $\prod_{t \le k} \deg(a_t) \le 6^k$, but each node costs only the size of its candidate set, and pruning keeps the explored nodes near-constant in k once the prefix is specific enough. Branch and bound has the same $O(4^k \cdot n)$ worst case as exhaustive search. The measured benefit is on nodes and not on time, because the bound |alive| is loose at shallow depth, where most positions are still alive and most of the filtering work happens.

Centre expansion is $O(n^2)$ in the worst case, with a measured slope of 2.02 on A^n, and Manacher is $O(n)$, with a slope of 1.04 on the same input. On real DNA both are effectively linear, at slopes 1.04 and 1.02, because palindromes are short. Manacher's advantage therefore only shows on adversarial input.

6 Results

6.1 Genome comparison

MN908947.3 (Wuhan) length	29903
MT106054.1 (Texas) length	29882
$\lvert LCS \rvert$	29875
similarity $\lvert LCS \rvert / \max(n, m)$	0.99906
substituted positions	6
synonymous	3
non-synonymous	3
outside coding regions	0
insertions / deletions / complex	1 / 2 / 0
Hirschberg time (alignment)	3.7 s
full DP table memory (not run)	3.6 GB

Table 2: Wuhan 2019 versus Texas 2020 by longest common subsequence.

$|LCS| = 29\,875$ against lengths 29 903 and 29 882 gives a similarity of 0.99906. The LCS counts the symbols that can be kept in order in both genomes, so $n - |LCS| = 28$ symbols of the Wuhan genome and $m - |LCS| = 7$ symbols of the Texas genome have no partner. The measure has one limitation: the LCS ignores locality, and a scattered set of matches scores the same as a contiguous block of the same size. This is why the gap analysis is needed to say where the differences are.

Wuhan pos	Texas pos	base	region	codon	amino acid	effect
8782	8782	C→T	QHD43415_4	AGC→AGT	S→S	synonymous
18603	18603	T→C	QHD43415_13	CAT→CAC	H→H	synonymous
18975	18975	T→A	QHD43415_13	GTT→GTA	V→V	synonymous
19175	19175	A→C	QHD43415_13	GAT→GCT	D→A	non-synonymous
27925	27925	C→T	QHD43422	ACA→ATA	T→I	non-synonymous
28144	28144	T→C	QHD43422	TTA→TCA	L→S	non-synonymous

Table 3: Substitutions between the two genomes (1-based positions) and their effect on the encoded amino acid.

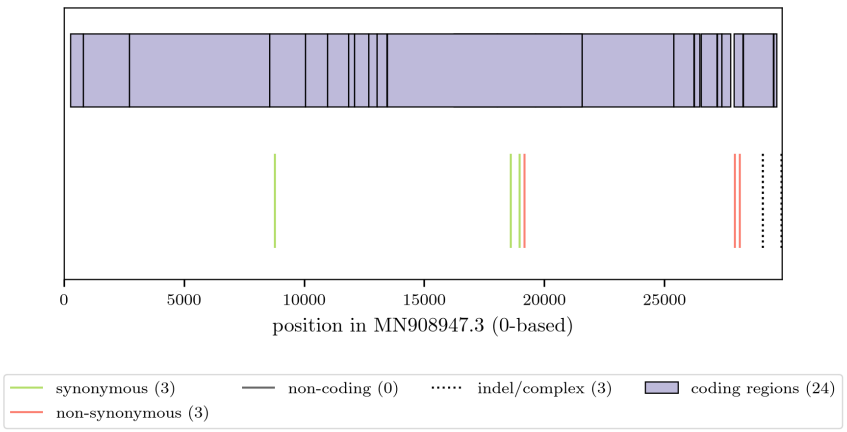


Figure 2: Position of every difference along the Wuhan genome, with the coding regions located by backtracking.

There are six substitutions: three of them synonymous and three not. The three non-synonymous ones are D→A in QHD43415_13 and two in QHD43422, T→I and L→S. The three indels are all after position 29 000. One is a 21-nucleotide run of A missing from the Texas record, which reflects how far the sequencing read went, not a mutation. The other two are an insertion and a deletion three positions apart near 29 091, and together they are really one more substitution, C→T at 29 095: the LCS has no substitution operation, and when the changed base equals its neighbour (Wuhan `GCTTTCGG`, Texas `GCTTTTGG`) an indel pair and a substitution give the same LCS length, so the traceback may return either. This is the main limitation of using the LCS as a diff. An edit-distance objective under the same Hirschberg recursion, which does have a substitution operation, would report the change at 29 095 as a substitution. Note that two substitutions also fell outside the three gene files and could only be classified after backtracking located QHD43422.

6.2 Search, greedy, backtracking, branch and bound, palindromes

Pattern search found all three genes at exactly the header coordinates: ORF1ab at 266, S at 21 563 and M at 26 523 in 1-based positions. In the Texas genome S and M are found intact while ORF1ab is not, which is consistent with the substitutions in Table 3.

Greedy activity selection took 187 candidate ORFs and selected 8, recovering S and M exactly and ORF1ab as its two large pieces, ORF1a and ORF1b. Lowering the threshold to 30 codons makes greedy pick 53 small ORFs that cover 30% of the genome and recover none of the known genes, because the objective is the count (Table 6, Figure 7).

Backtracking located 23 of the 24 proteins over their full length, with between 19 and 143 nodes each. The exception, QHD43415_11, matches exactly 9 amino acids from position 13 441, that is, up to nucleotide 13 468, the ribosomal frameshift, so the algorithm found the biology rather than failing (Table 5).

Branch and bound found, for $k = 8$ and $d = 1$, the motif TGTTGTTA with 75 occurrences, the same optimum as exhaustive search, using 6 383 nodes instead of 85 695 (Table 7).

Manacher gave a longest palindrome of 11, 15 and 20 nucleotides in M, S and ORF1ab (Table 8).

6.3 Performance

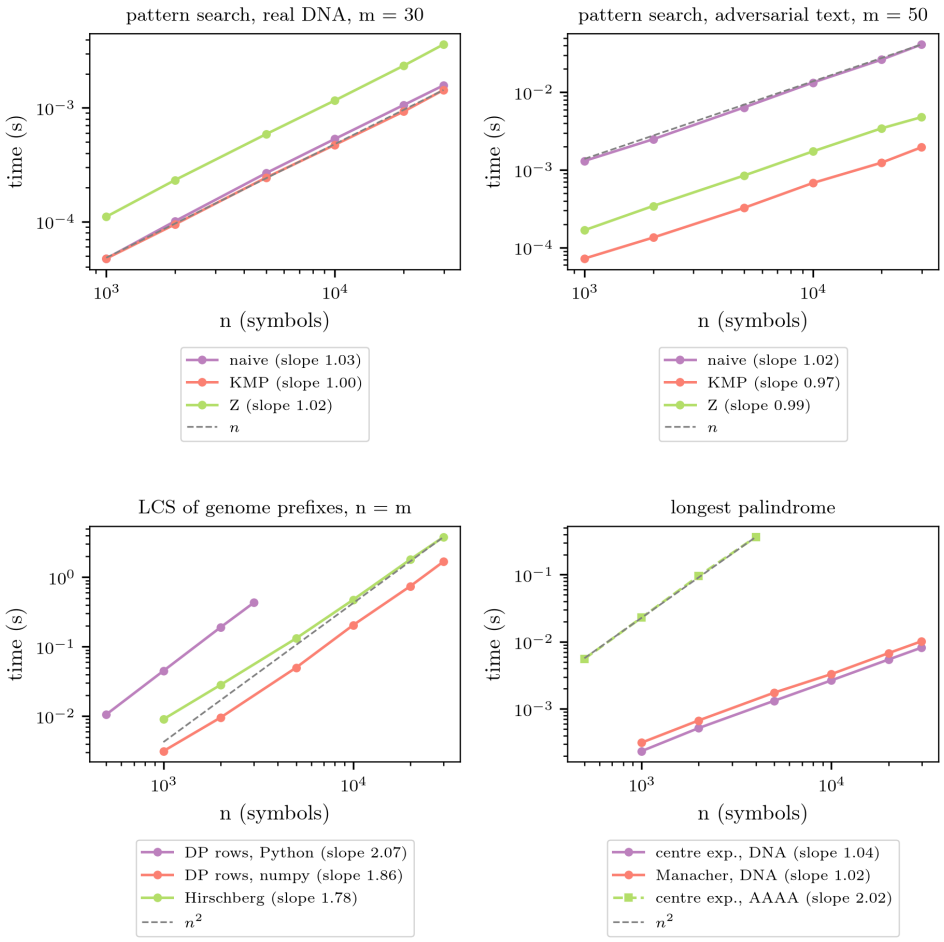


Figure 3: Running time against input size (log-log). Dashed lines are the theoretical growth anchored at the largest measurement; slopes in the legends are least-squares fits.

On real DNA the three search methods are all linear, at slopes 1.03, 1.00 and 1.02 (Table 4). Naive search is not slower than the others because a mismatch ends most alignments after one or two comparisons. The Z-function is two to three times KMP because it builds and stores an array of $n + m + 1$ values while KMP keeps one pointer. On the adversarial text naive search still has slope 1 in n, but it is about 20 times slower than KMP, the factor $m = 50$ of the $O(nm)$ bound; KMP and Z are unchanged. In the LCS panel the pure Python rows and the numpy rows both have a slope close to 2, quadratic as $O(nm)$ predicts with $n = m$, and Hirschberg has the same slope and costs about twice the numpy rows, the $2nm$ bound; the pure Python version reaches only $n = 3\,000$, where one alignment already takes 431.5 ms, while numpy covers the full 29 903 in 1.68 s. In the palindrome panel centre expansion is linear on DNA (1.04) but quadratic on A^n (2.02), while Manacher is linear on both.

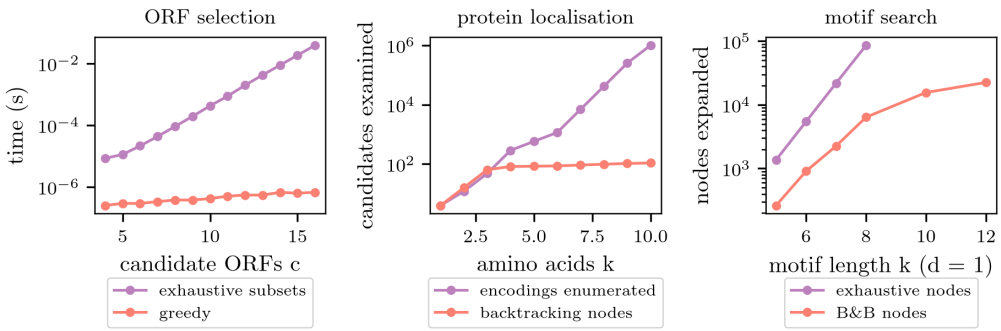


Figure 4: Search-space techniques against their baselines: exhaustive subsets versus greedy, encodings enumerated versus backtracking nodes, exhaustive versus branch-and-bound nodes.

Exhaustive subset search doubles with every candidate while greedy stays flat. Enumerating encodings grows by the codon degeneracy at each amino acid and reaches 995 328 at $k = 10$, while backtracking stays near 100 nodes, 108 at the same k, because after three amino acids the prefix pins down a single genome position. Branch and bound expands about 7% of the exhaustive nodes at $k = 8$, 6 383 against 85 695, and keeps growing slowly to $k = 12$, where exhaustive search is no longer feasible.

7 Conclusions

Dynamic programming gives the exact answer, but its table is the bottleneck. Divide and conquer did not change the time bound, it changed what fits in memory, and that is what made the full-genome comparison possible. The linear-time string algorithms only pay off when the pattern is repetitive: on real DNA brute force is already linear in practice, so the theoretical worst case is what justifies KMP, not the average run. Greedy is optimal for the objective it is given, and the ORF experiment shows that the objective is the whole story, since maximum count and maximum coverage are different problems. Backtracking turns an exponential enumeration into a few dozen nodes because the genome is a very strong constraint, and its one apparent failure was the frameshift, a fact about the data. Branch and bound is only as good as its bound: the same optimum with far fewer nodes, but no time saved, because the bound is loose where the work is. Manacher is the only technique whose advantage was invisible on the real data and visible only under an adversarial input.

Comparing the techniques, the linear string algorithms and greedy scaled best, while the LCS and the exhaustive baselines needed the most resources. The datasets changed the behaviour: real DNA against adversarial input flips both the naive-versus-KMP and the centre-versus-Manacher comparisons, and the length threshold flips the greedy result. Hirschberg is the best approach here, on one criterion: it is the only technique that made an infeasible problem feasible, turning a 3.6 GB table into linear space and so allowing the two genomes to be compared at all. Backtracking had the largest gain over its baseline, 108 nodes against 995 328 encodings at $k = 10$, but its baseline could still be run.

Relating experiment to theory worked in three ways. The log-log slopes reproduce the exponents of the complexity bounds, so the measurements confirm the analysis instead of replacing it. Constants matter as well as exponents: numpy against pure Python, and the $2nm$ of Hirschberg against the nm of the length-only row. And the baseline has to be run to know whether the sophisticated algorithm was needed at all.

References

Anthropic (2026). Claude. Large language model, `https://claude.ai`. Used to assist with code and with the redaction of this report.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. MIT Press, 3rd edition.

Hirschberg, D. S. (1975). A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, 18(6):341–343.

Knuth, D. E., Morris, J. H., and Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350.

Manacher, G. (1975). A new linear-time on-line algorithm for finding the smallest initial palindrome of a string. *Journal of the ACM*, 22(3):346–351.

A Additional tables

algorithm	technique	complexity	slope real	slope adv.	largest run (real / adv.)
naive search	brute force	$O(nm)$	1.03	1.02	1.6 ms / 41.5 ms, $n = 29903$
KMP	string	$O(n+m)$	1.00	0.97	1.4 ms / 2.0 ms, $n = 29903$
Z-function	string	$O(n+m)$	1.02	0.99	3.6 ms / 4.8 ms, $n = 29903$
LCS rows, Python	DP	$O(nm)$ time, $O(m)$ space	2.07	–	431.5 ms, $n = 3000$
LCS rows, numpy	DP	$O(nm)$ time, $O(m)$ space	1.86	–	1.68 s, $n = 29903$
Hirschberg	D&C + DP	$O(nm)$ time, $O(n+m)$ space	1.78	–	3.79 s, $n = 29903$
ORF selection	greedy	$O(n + c \log c)$ vs $O(2^c)$	1.01 (scan)	–	exhaustive 38.6 ms, $c = 16$
protein localisation	backtracking	$O(6^k)$ worst, pruned	–	–	108 vs 995328 nodes, $k = 10$
motif search	branch and bound	$O(4^k n)$ worst, pruned	–	–	6383 vs 85695 nodes, $k = 8$
centre expansion	brute force	$O(n^2)$	1.04	2.02	8.2 ms / 364.1 ms
Manacher	string	$O(n)$	1.02	1.04	10.2 ms / 1.6 ms

Table 4: Theoretical complexity against the measured log-log slope of running time versus input size. Timings from one run of the notebook on a laptop.

protein	length	start	end	frame	matched	nodes	pruned
QHD43415_1	180	265	805	1	180/180	41	32
QHD43415_2	638	805	2719	1	638/638	86	65
QHD43415_3	1945	2719	8554	1	1945/1945	76	56
QHD43415_4	500	8554	10054	1	500/500	41	27
QHD43415_5	306	10054	10972	1	306/306	97	66
QHD43415_6	290	10972	11842	1	290/290	140	103
QHD43415_7	83	11842	12091	1	83/83	56	31
QHD43415_8	198	12091	12685	1	198/198	86	64
QHD43415_9	113	12685	13024	1	113/113	42	30
QHD43415_10	139	13024	13441	1	139/139	68	45
QHD43415_11	932	13441	16237	1	9/932	90	58
QHD43415_12	601	16236	18039	0	601/601	102	76
QHD43415_13	527	18039	19620	0	527/527	42	26
QHD43415_14	346	19620	20658	0	346/346	142	87
QHD43415_15	298	20658	21552	0	298/298	143	93
QHD43416	1273	21562	25381	1	1273/1273	25	17
QHD43417	275	25392	26217	0	275/275	26	18
QHD43418	75	26244	26469	0	75/75	29	21
QHD43419	222	26522	27188	2	222/222	37	26
QHD43420	61	27201	27384	0	61/61	23	15
QHD43421	121	27393	27756	0	121/121	23	15
QHD43422	121	27893	28256	2	121/121	19	12
QHD43423	419	28273	29530	1	419/419	31	19
QHI42199	38	29557	29671	1	38/38	22	13

Table 5: Proteins located by backtracking with $k = 6$ amino acids (0-based genome positions). Matched is the number of leading amino acids reproduced by translating the genome from the located start.

start	end	frame	codons	known gene overlap
265	13483	1	4405	ORF1ab (62%)
13767	21555	0	2595	ORF1ab (37%)
21562	25384	1	1273	S (100%)
25392	26220	0	275	-
26522	27191	2	222	M (100%)
27393	27759	0	121	-
27893	28259	2	121	-
28273	29533	1	419	-

Table 6: ORFs selected by greedy activity selection among candidates with at least 100 codons (0-based positions).

k	d	best motif	count	exhaustive nodes	B&B nodes
5	1	TTTTT	1241	1365	263
6	1	TTTTTT	418	5461	907
7	1	TGTTGTT	193	21836	2252
8	1	TGTTGTTA	75	85695	6383
10	1	AAAAGAAAAA	33	–	15575
12	1	AAAAAAAAAAAA	24	–	22445

Table 7: Most frequent k-mer with up to one mismatch in the Wuhan genome.

gene	length	longest palindrome	start	end	length
M	669	`CTAAAGAAATC`	493	504	11
S	3822	`GAATTTGTGTTTAAG`	570	585	15
orf1ab	21290	`CTCAATGACTTCAGTAACTC`	9711	9731	20

Table 8: Longest palindrome per gene (0-based positions inside the gene).

B Additional figures

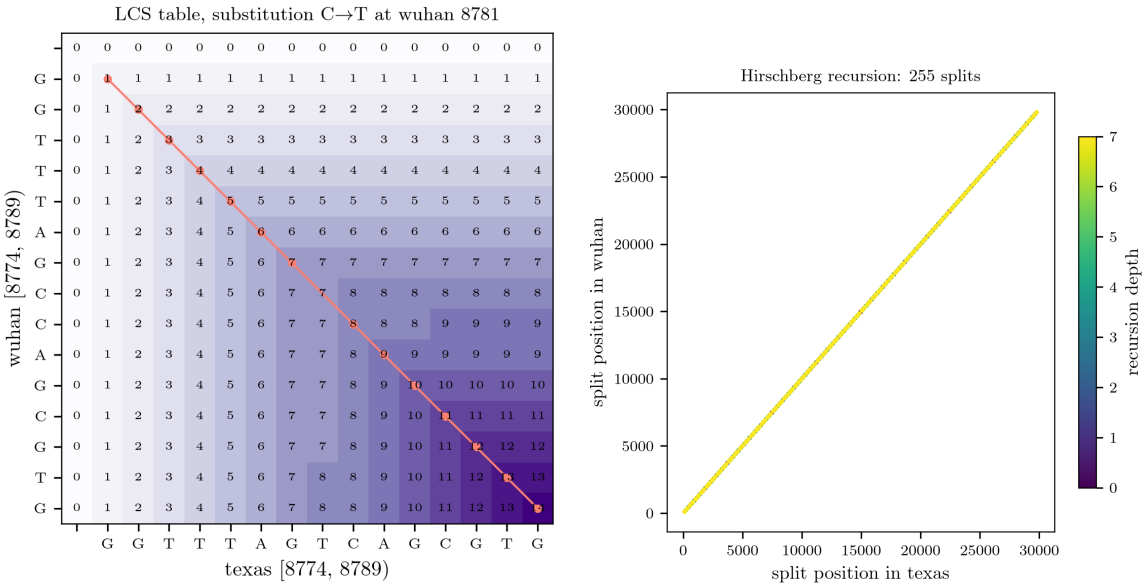


Figure 5: Left: LCS table and traceback on a 15-symbol window around the first substitution. Right: the split points chosen by Hirschberg's recursion on the two genomes, coloured by depth.

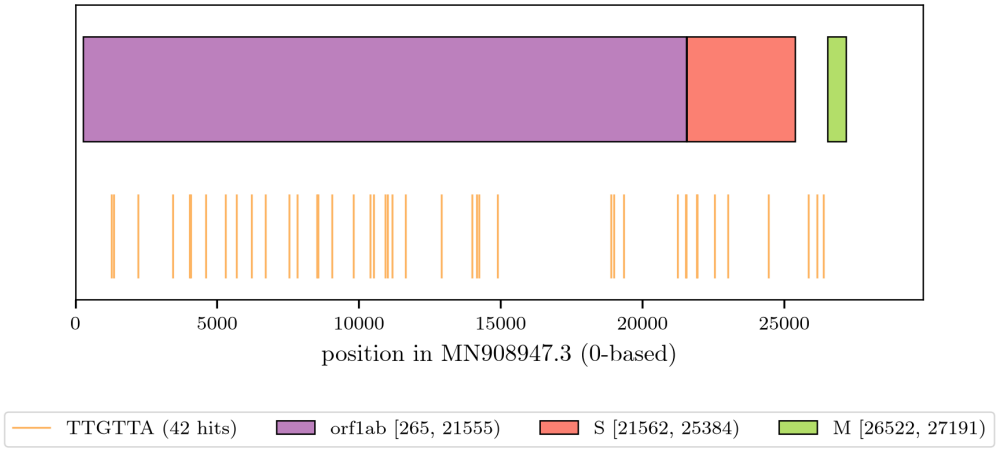


Figure 6: Genes located by pattern search in the Wuhan genome and the occurrences of the most frequent 6-mer.

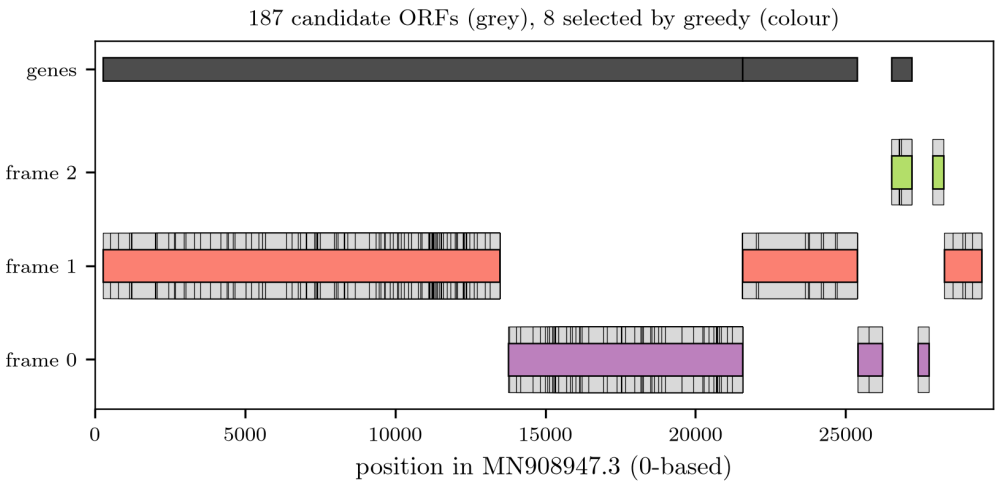


Figure 7: Candidate ORFs per reading frame (grey) and the greedy selection (colour) against the known genes.

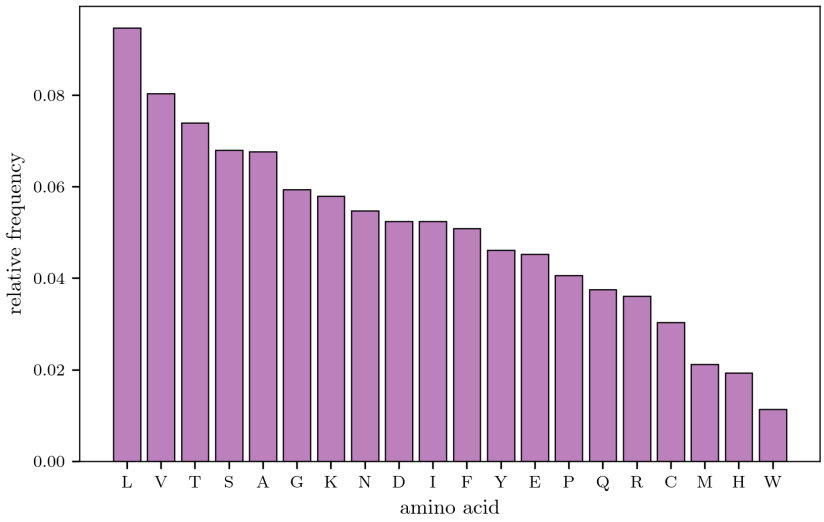


Figure 8: Pooled amino-acid frequencies over the 24 protein records.